SECURITY TESTING

ASSIGNMENT 3

Tran Dinh Hai - s4041605

Lecturer:
Dr. Sreenivas Tirumala

Table of Contents

1. Exploiting Web Services using SQL Injection Attacks

Q1.1 In the above SQL statement, the variable `$input_uname` holds the string typed in the Username textbox, and `$hashed_pwd` holds the string typed in the Password textbox. User's inputs in these two textboxes are placed directly in the SQL query string. There is a SQL-injection vulnerability in the above SELECT query. Exploit the vulnerability and log into Ted's account without knowing the correct password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_home.php`, is:

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
nickname, Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn -> query($sql);
```

Answer

Vulnerability analysis. Both values placed into `$sql` originate from the client and are never validated. `$input_uname` is read unmodified from `$_GET['username']`, and `$hashed_pwd` is the SHA-1 digest of `$_GET['Password']`. Hashing the password does not protect this statement: SHA-1 only transforms the *password* field into a fixed-length hexadecimal string, but the vulnerability lies in the neighbouring `name` column, where `$input_uname` is concatenated into the query completely unmodified. Whatever the password field contains, an attacker who controls the `username` field alone can still reshape the statement. Both variables are joined into the query by direct PHP string interpolation, with no parameterised query (an `mysqli` prepared statement with bound placeholders) and no input escaping (`mysqli_real_escape_string()`) applied to either one:

```
WHERE name= '$input_uname' and Password='$hashed_pwd'
```

The single quotes around each variable mark where the developer *intends* the string literal to start and end, but they do nothing to stop the value itself from containing a quote character. Because the query is assembled as plain text before MySQL ever parses it, a quote inside `$input_uname` is indistinguishable from the quote the developer typed around the variable. This is the defining trait of a string-based SQL injection in an authentication check: user input is interpreted as part of the query's *syntax* rather than as inert *data*, so whoever controls the input can rewrite the logic of the query, including removing the password predicate entirely.

Exploiting the vulnerability. I submitted `Ted'#` in the Username field and left the Password field blank, as shown in Figure 1.1. The browser transports the # character in the request line as the percent-encoded sequence `%23`; the web server decodes this back to the literal # before PHP ever reads `$_GET['username']`, so the value that reaches `$sql` is the raw character, not the encoded form shown in the address bar.

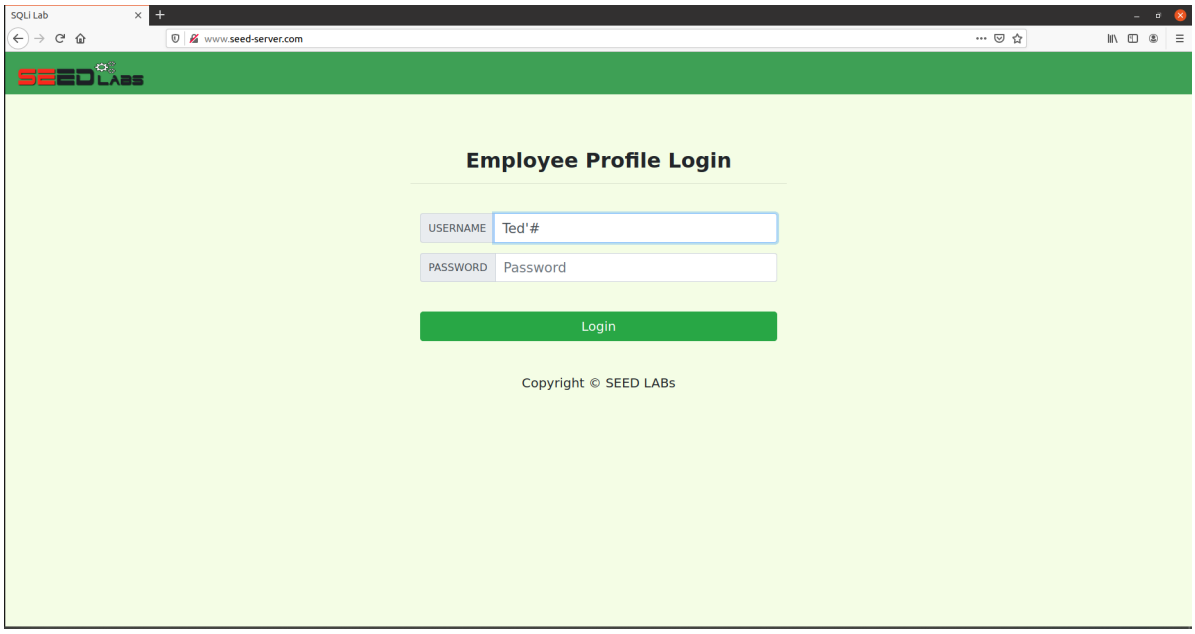

Figure 1.1. The injected username entered with the password field left blank.

With the Password field empty, `$hashed_pwd` is `sha1("")`. Substituting `Ted'#` for `$input_uname` therefore produces the raw statement, in which `$hashed_pwd` appears as its computed SHA-1 digest:

```
1  WHERE name = 'Ted'#' and Password='da39a3ee5e6b4b0d3255bfef95601890afd80709'
```

MySQL parses this left to right. `Ted` selects the intended account name. The apostrophe immediately after it is the one I injected: it closes the string literal that the developer opened with `name= '`, so the database compares `name` against the literal `Ted` only, not `Ted'#`. Parsing then reaches the # character. In MySQL, # begins a single-line comment that extends to the end of the statement, so everything after it – the stray closing quote left over from the original template, the literal `and Password=`, and the SHA-1 hash of the blank password field – is discarded before execution instead of being treated as SQL. The effective condition MySQL actually evaluates is therefore reduced to:

```
1  WHERE name = 'Ted'
```

This selects Ted's row by name alone; no password predicate participates in the query at all. In `unsafe_home.php`, a non-empty `$id` in the returned row is treated as a successful authentication: the script starts a session, stores Ted's identity in it, and calls `drawLayout()` to render his profile. Figure 1.2 confirms this outcome: the application displays the Ted Profile page populated with Employee ID 50000 and Salary 110000, the values recorded for Ted in `sqllab_users.sql`,

confirming that the returned row genuinely belongs to Ted rather than to a different or empty account.

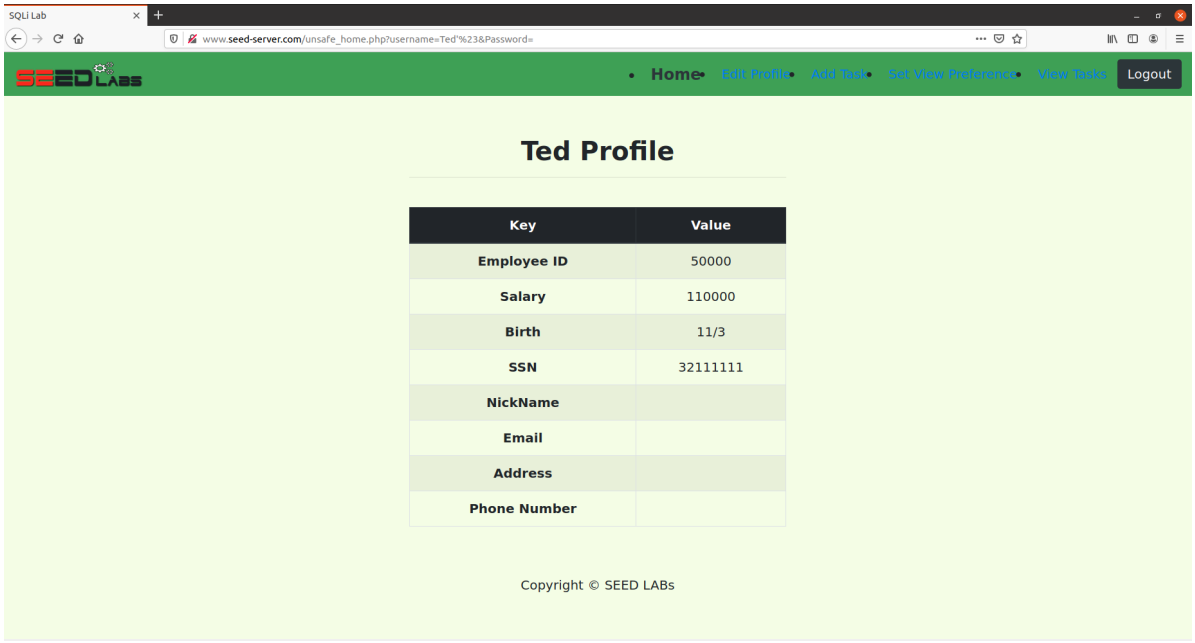

Figure 1.2. Ted's profile displayed after the injected login request.

This demonstrates that the login succeeded without knowledge of Ted's correct password: the SHA-1 comparison that should have enforced it was never evaluated, because the injected comment removed it from the executed query before MySQL had a chance to check it.

Q1.2 In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Boby's salary to SALARY_1 without knowing Ted's password and Boby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

The code snippet given in the assignment brief for this sub-question, taken from `unsafe_edit_backend.php`, is:

```
$hashed_pwd = sha1($input_pwd);
$sql = "UPDATE credential SET
    nickname='$input_nickname',
    email='$input_email',
    address='$input_address',
    Password='$hashed_pwd',
    PhoneNumber='$input_phonenumber'
WHERE ID=$id;";
$conn->query($sql);
```

Answer

Vulnerability analysis. `$id` is read from `$_SESSION['id']`, so an attacker cannot set it directly through a request parameter – it is always the ID of whoever is currently logged in, Ted (`ID=5`) in this case. However, `NickName`, `Email`, `Address`, `Password`, and `PhoneNumber` are all read unmodified from `$_GET` and concatenated into `$sql` with no parameterised query and no escaping, exactly as in `unsafe_home.php`. Because the single quotes around each of these fields only mark the developer's *intended* string boundary, an attacker who breaks out of any one of them can append arbitrary SQL, including a brand-new `WHERE` clause, and comment away the original one. This means the attacker does not need to control `$id` at all: redirecting the `WHERE` clause is enough to make the whole statement apply to any row, regardless of who is logged in.

The full snippet above is only executed when the Password field is non-empty. The actual file wraps it in a condition:

```
if($input_pwd!=''){
    $hashed_pwd = sha1($input_pwd);
    $_SESSION['pwd']=$hashed_pwd;
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',Password='$hashed_pwd',PhoneNumber='$input_phonenumber
    ' where ID=$id;";
}else{
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
}
$conn->query($sql);
```

I left the Password field empty, so the executed statement is built by the `else` branch, which omits the `Password` assignment entirely. This branch is also never checked for success: `$conn->query($sql)` is called and the script unconditionally redirects to `unsafe_home.php` whether the query succeeded or failed, so the application itself gives no on-page indication of the outcome.

Exploiting the vulnerability. While logged in as Ted from Q1.1, I opened Edit Profile and entered the following value in the NickName field, leaving Email, Address, and Phone Number at their pre-filled (blank) values and Password empty, as shown in Figure 1.3:

```
', Salary=1605 WHERE ID=2 --
```

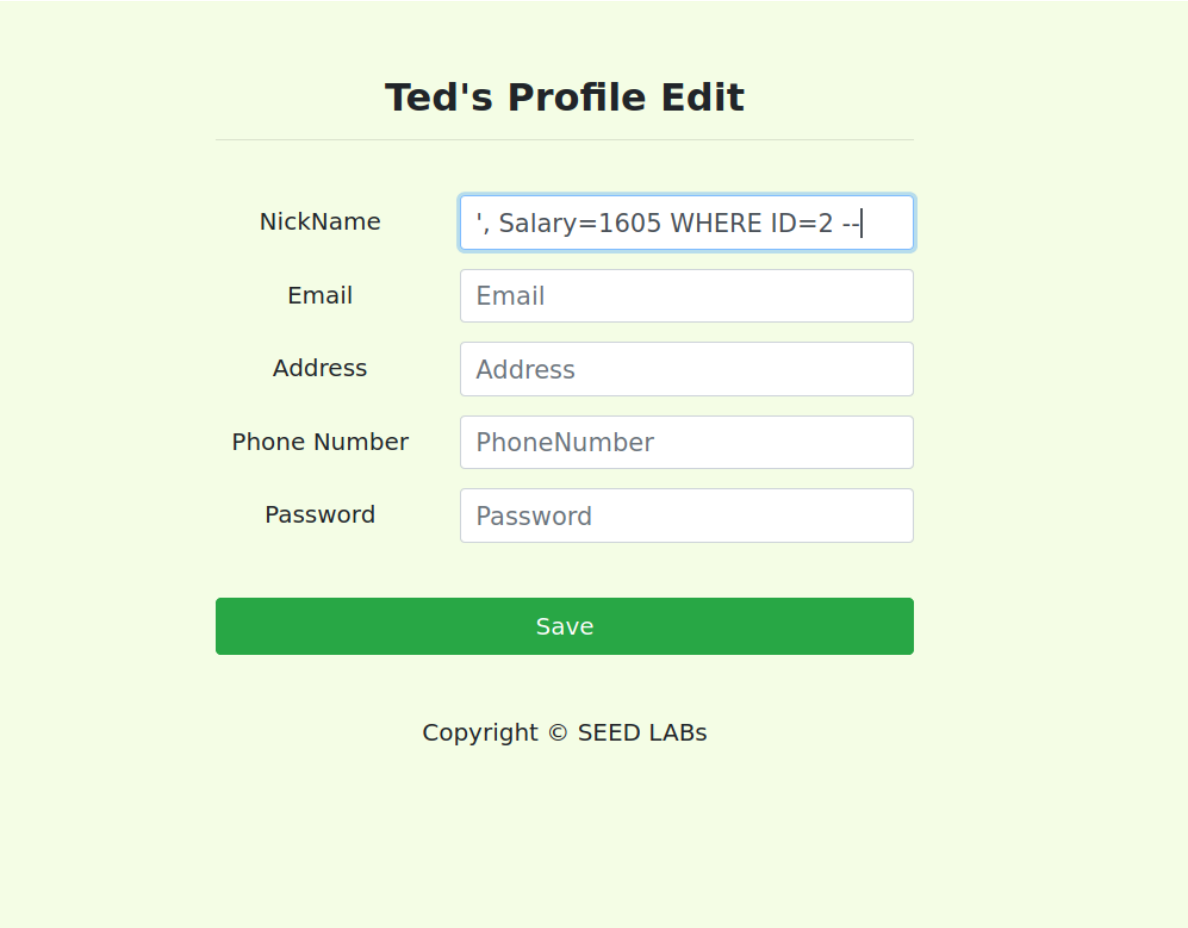

Figure 1.3. The injected NickName payload entered on Ted's Edit Profile form.

Substituting this into the `else` branch's `$sql`, with Ted's session `$id` equal to `5`, produces the raw statement:

```
1  UPDATE credential SET nickname='', Salary=1605 WHERE ID=2 -- ',email='',address='',
       PhoneNumber='' where ID=5;
```

The leading apostrophe in the payload closes the `nickname='...'` literal immediately, giving `nickname=''` – a harmless no-op, since Boby's NickName is already empty in the seed data. Everything after that quote is now parsed as SQL rather than string content: `, Salary=1605` adds a second column assignment, and `WHERE ID=2` supplies a brand new condition that targets Boby instead of Ted. The trailing `–`, followed by a space, then opens a MySQL end-of-line comment that discards everything after it – the stray closing quote left over from the template, the remaining `email`/`address`/`PhoneNumber` assignments, and the real `where ID=5` clause that would otherwise have kept the update on Ted's own row. The effective statement MySQL actually executes is therefore reduced to:

```
1  UPDATE credential SET nickname='', Salary=1605 WHERE ID=2
```

This updates Boby's row (`ID=2`) alone; Ted's row is never touched, because the real `where ID=5` clause was commented out before MySQL parsed it.

Since `unsafe_edit_backend.php` gives no visible success or failure message, I verified the result out of band by logging out and logging back in as Boby using the same technique as Q1.1 (`Boby'#` with the Password field left blank), which loads Boby's profile directly from his row in `credential`. Figure 1.4 shows the result:

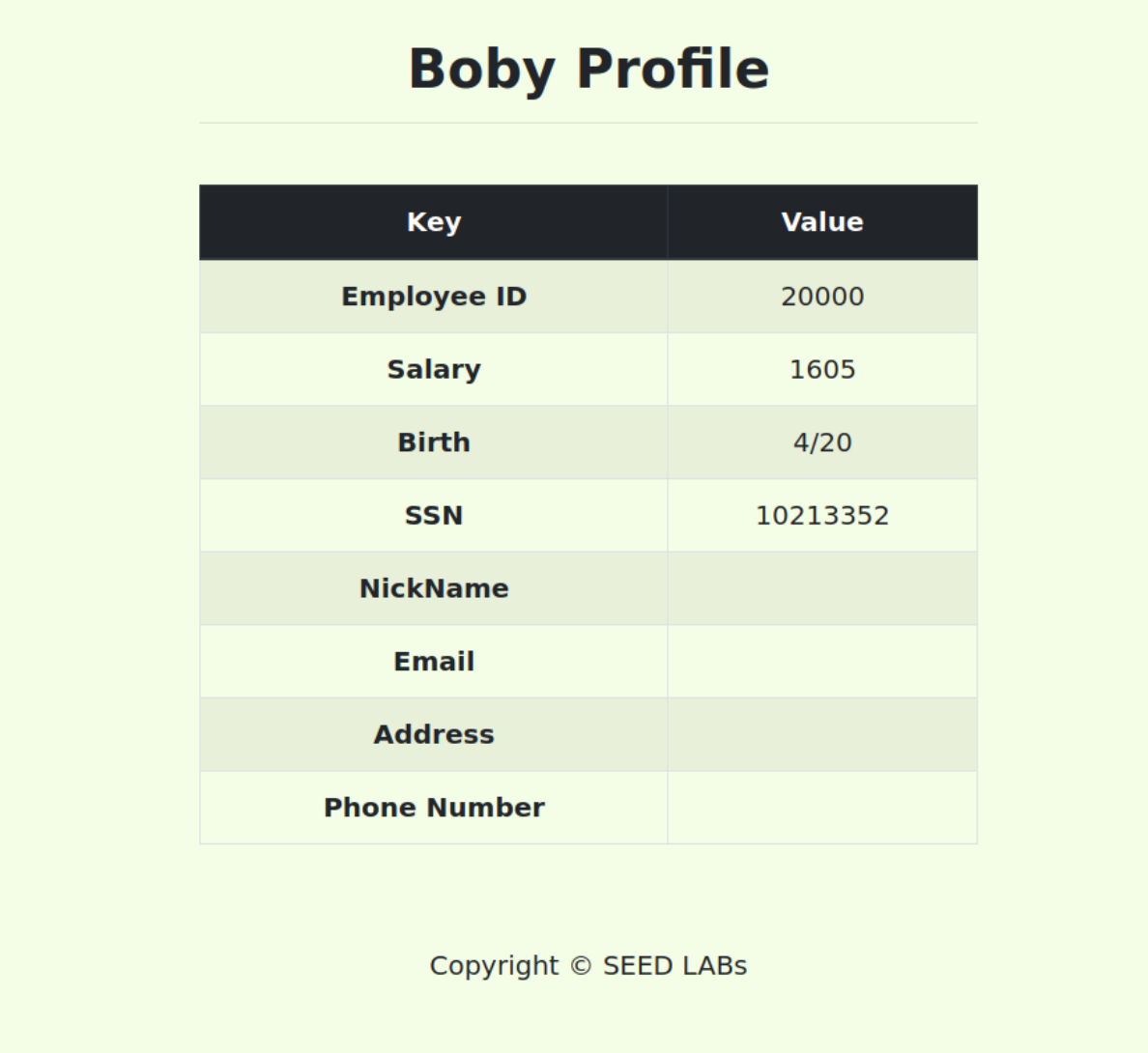

Figure 1.4. Boby's profile after the attack, showing Salary updated to 1605.

Boby's Salary field now reads `1605`, matching SALARY_1 for student s4041605. Employee ID (20000), Birth (4/20), and SSN (10213352) are unchanged from `sqllab_users.sql`, confirming this is genuinely Boby's row, and NickName remains blank, consistent with the injected `nickname=''` assignment being a no-op. This demonstrates that Ted's authenticated session was used to modify a different user's row without knowing Ted's password (already bypassed in Q1.1) or Boby's password at any point.

Q1.3 In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Boby's nickname to your student name, email to your RMIT student email, and address to "RMIT", without knowing Ted's password and Boby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Answer

Vulnerability analysis. This sub-question exploits the same `UPDATE` statement identified in Q1.2, again via the `else` branch of `unsafe_edit_backend.php` that runs when the Password field is left empty:

```
$sql = "UPDATE credential SET nickname='$input_nickname',email='$input_email',
    address='$input_address',PhoneNumber='$input_phonenumber' where ID=$id;";
```

In Q1.2 I broke out of the *first* field (`NickName`) and commented away everything after it, so the earlier, genuine-looking `email`/`address`/`PhoneNumber` values were never applied to any row – only the injected `Salary` assignment mattered. For this sub-question I need three fields (`NickName`, `Email`, `Address`) to actually take effect on Boby's row, so injecting through the first field will not work: anything before the injection point is discarded by the comment, and anything after it never executes on the intended row. Instead, I inject through the *last* field, `PhoneNumber`, immediately before the real `WHERE` clause. A single `UPDATE` statement has exactly one `WHERE` clause governing every `SET` assignment in it, so replacing that clause at the last possible point makes *all* of the preceding assignments – including the legitimate-looking NickName, Email, and Address values – apply to whichever row the injected clause names, rather than to Ted's own row.

Exploiting the vulnerability. While logged in as Ted, I opened Edit Profile and entered the following values, leaving Password empty so the `else` branch above is used, as shown in Figure 1.5:

- NickName: `Tran Dinh Hai`
- Email: `s4041605@rmit.edu.vn`
- Address: `RMIT`
- Phone Number: `' WHERE ID=2#`

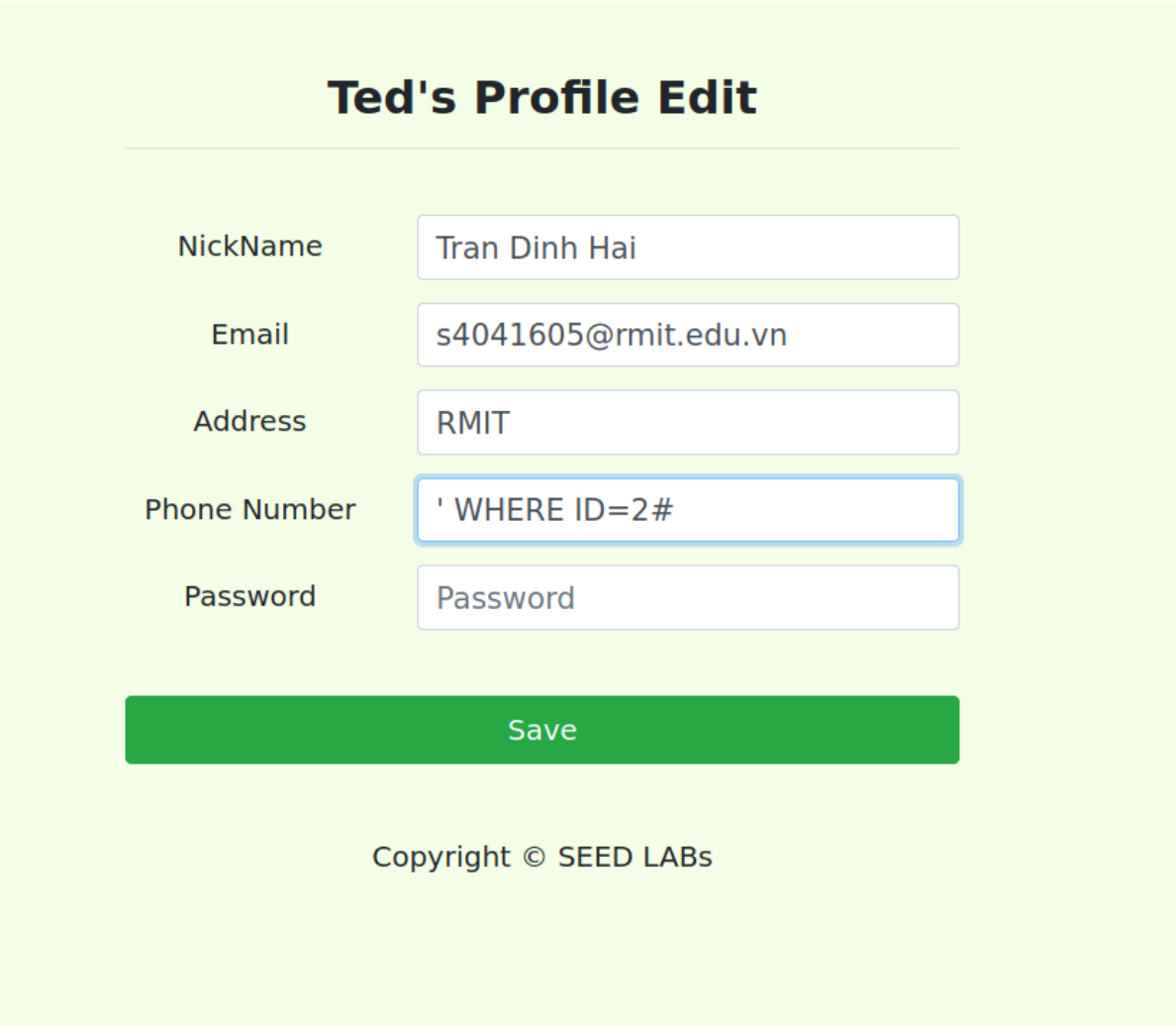

Figure 1.5. The injected values entered on Ted's Edit Profile form, with the payload in Phone Number.

Substituting these into `$sql`, with Ted's session `$id` equal to `5`, produces the raw statement:

```
UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
    ='RMIT',PhoneNumber='' WHERE ID=2#' where ID=5;
```

The apostrophe in the Phone Number payload closes `PhoneNumber='...'` immediately, giving `PhoneNumber=''` – a no-op, since Boby's Phone Number is already blank in the seed data. `WHERE ID=2` then supplies a brand-new condition for the statement, and `#` opens a MySQL comment that discards everything after it: the stray closing quote left over from the template and the real `where ID=5` clause. Because this substitution replaces the statement's only `WHERE` clause rather than being commented out itself, every assignment before it – `nickname`, `email`, and `address` – is retained and now applies to `ID=2`. The effective statement MySQL actually executes is:

```
UPDATE credential SET nickname='Tran Dinh Hai',email='s4041605@rmit.edu.vn',address
    ='RMIT',PhoneNumber='' WHERE ID=2
```

I verified the result the same way as Q1.2: logging out and back in as Boby (`Boby'#`, Password blank). Figure 1.6 shows the result:

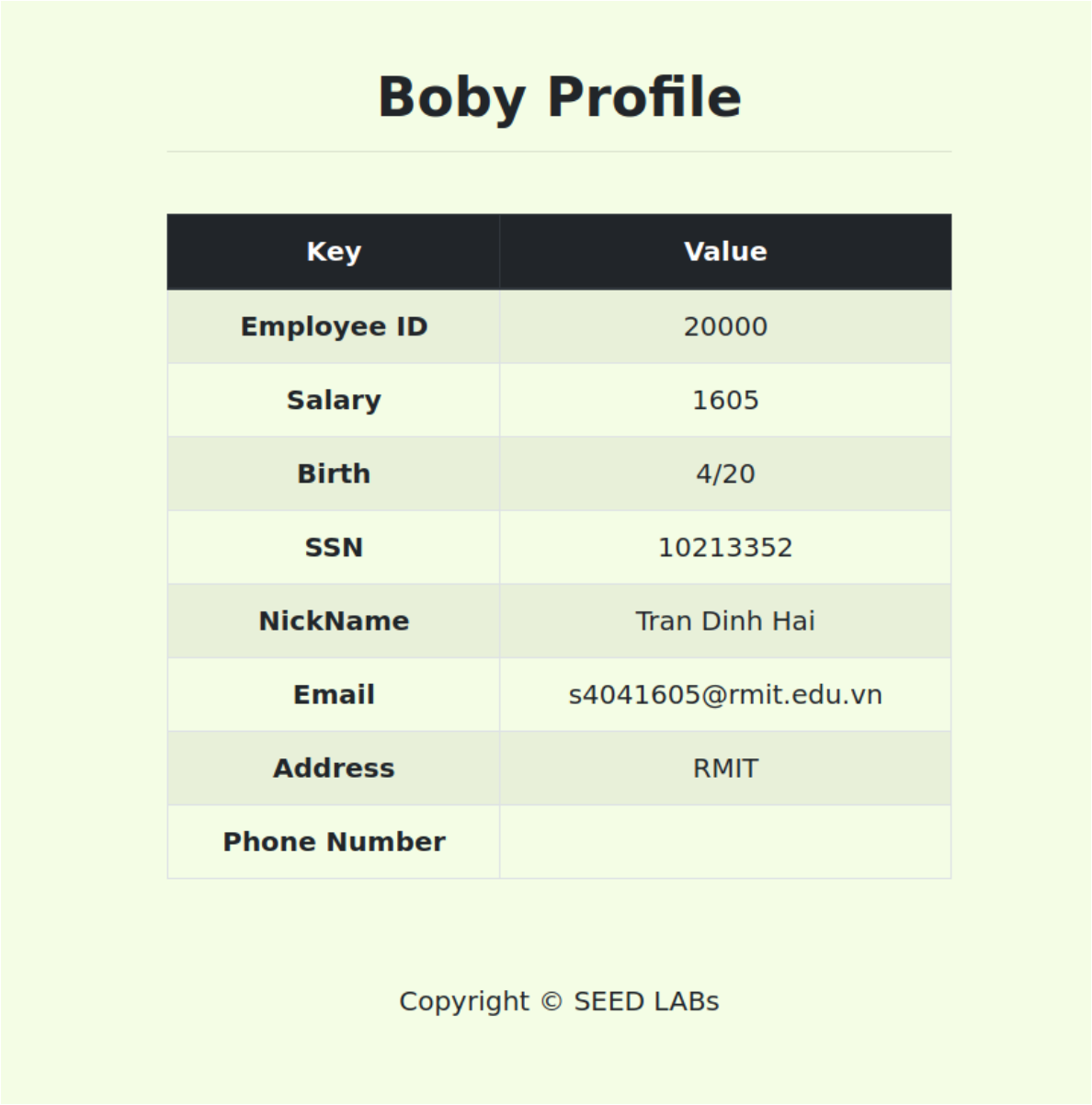

Figure 1.6. Boby's profile after the attack, showing the updated NickName, Email, and Address.

Boby's profile now shows NickName `Tran Dinh Hai`, Email `s4041605@rmit.edu.vn`, and Address `RMIT`, exactly as submitted. Phone Number remains blank, consistent with the injected `PhoneNumber=''` assignment being a no-op. Salary still reads `1605` from Q1.2, and Employee ID (20000), Birth (4/20), and SSN (10213352) remain unchanged, confirming this is still genuinely Boby's row. This demonstrates that Ted's authenticated session was again used to modify Boby's row – this time three fields at once – without knowing Ted's password or Boby's password at any point.

Q1.4 In the above two web pages, you need to identify the vulnerable SQL queries that are exploitable to allow the attacker to conduct the above attack: "display all the tasks of the user who has the maximum number of tasks when you view your tasks". Show the identifies vulnerabilities and explain why they are exploitable to allow for the attack.

Answer

TODO *Identify the vulnerable queries in `unsafe_view_order.php` and `unsafe_tasks_view.php`, trace attacker-controlled data into them, and add genuine source evidence.*

FIGURE PLACEHOLDER

Vulnerable task queries identified for Q1.4.

Figure 1.7. Vulnerable task queries identified for Q1.4.

Q1.5 You need to exploit the identifies vulnerable SQL queries in above Q1.5 to conduct the attack: "display all the tasks of the user who has the maximum number of tasks when you view your tasks". Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Answer

TODO *Record the exact injected input and resulting SQL statements. Add genuine evidence identifying the user returned by `userIdMaxTasks()` and showing that the View Tasks page displays that user's tasks, then explain the complete data flow.*

FIGURE PLACEHOLDER

Successful display of the maximum-task user's tasks for Q1.5.

Figure 1.8. Successful display of the maximum-task user's tasks for Q1.5.

Appendix

Supporting Material

TODO *Add only supporting commands, configurations, or evidence that the Assignment 3 brief permits in an appendix. Remove this appendix if it is not required.*